



Table of Contents

Introduction	3
Contents.....	4
General Requirements.....	5
Instructions for Competitors.....	6
Entity Relationship Diagram	7
Authorization	8
1. Auth.....	9
POST /api/Auth/SignIn.....	9
GET /api/Auth/Me	10
2. Cart.....	12
GET /api/Transaction/{uniqueCode}/Cart	12
POST /api/Transaction/{uniqueCode}/Cart	13
DELETE /api/Transaction/{uniqueCode}/Cart/{itemId}	14
POST /api/Transaction/{uniqueCode}/Cart/Order	15
3. Items	17
GET /api/Items	17
4. Transaction.....	19
POST /api/Transaction	19
GET /api/Transaction/{uniqueCode}/Orders.....	20
POST /api/Transaction/{uniqueCode}/Pay	21
PUT /api/Transaction/{uniqueCode}/Orders/{orderId}/Items/{itemId}/Status.....	22

Introduction

Sakura Sushi, a renowned sushi restaurant in the heart of the city, has decided to embrace the digital age by developing a web API for their food ordering system. The restaurant, known for its exquisite sushi rolls and authentic Japanese cuisine, aims to enhance the customer experience by providing a seamless and efficient way to place orders online. With the increasing demand for online services, Sakura Sushi's management team recognized the need to modernize their ordering process to cater to tech-savvy customers who prefer the convenience of digital transactions.

The new web API will allow customers to browse the menu, and place orders. This system is designed to integrate with the restaurant's existing infrastructure, ensuring that orders are promptly received and processed by the kitchen staff. By leveraging modern web technologies, Sakura Sushi aims to reduce wait times, minimize order errors, and provide real-time updates to customers about their order status. The API will also support user authentication, allowing customers to create accounts, save their favorite orders, and access exclusive promotions.

To ensure the security and reliability of the system, the development team has implemented robust authentication mechanisms using JWT (JSON Web Tokens). This will safeguard customer data and ensure that only authorized users can access the system. Additionally, the API will be designed with scalability in mind, allowing Sakura Sushi to handle a growing number of online orders without compromising performance. The team is also working on integrating feedback mechanisms, enabling customers to rate their dining experience and provide valuable insights for continuous improvement.

With the launch of the web API, Sakura Sushi is poised to revolutionize its service delivery, offering a blend of traditional culinary excellence and modern convenience. The restaurant's commitment to quality and customer satisfaction remains unwavering, and this new initiative is a testament to their dedication to staying ahead in the competitive food industry. By embracing technology, Sakura Sushi not only enhances its operational efficiency but also strengthens its connection with a broader customer base, ensuring that more people can enjoy their delectable offerings with ease.

You, as a developer are contracted to build the web api application based on the details below.

Contents

- | | |
|---------------------------|-----------------------------|
| 1. LKSN2024_WebAPI_TP.pdf | (Test project description) |
| 2. Web API Folder | (Web API folder Example) |
| 3. sakurasushi.sql | (Database script) |
| 4. ERD.pdf | (Entity Relational Diagram) |

General Requirements

- Project Criteria
 - The project name should be in this format: “WebAPI_XX”, where XX is your workstation number.
 - Time management is a crucial thing for the success of this project. So, mind the given time and submit all your deliverables on time.
- Validation and Error Messages
 - Your application should work properly and show meaningful error messages.
 - Analyze the validation that needs to be implemented, as this is one of the scoring criteria.
 - Any specific validation will be provided on the test project.
- Default Format
 - The de-facto standard, ISO compliant date and time format is YYYY-MM-DDThh:mm:ssZ

Instructions for Competitors

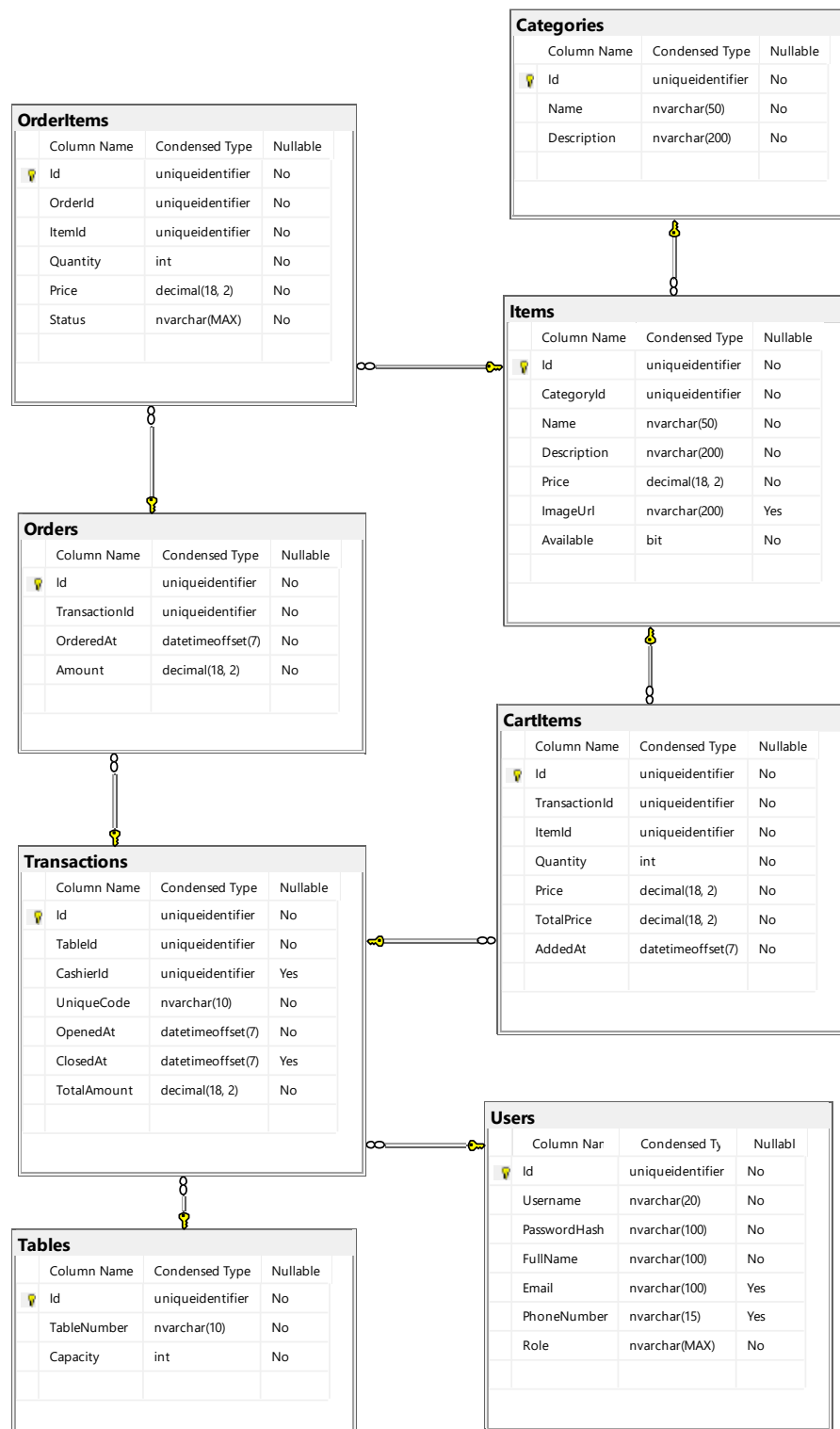
1. Utilize the Web API

For this test project, you are going to use the Web API given. You can access the web api through this link: <http://localhost:5555/swagger> .

2. Create the application and submit answer

Create the requested web api application based on the detail description on the next part of this test project. Submit all your answer/deliverables by compressing all your works (.zip) and upload it to the provided link.

Entity Relationship Diagram



To help further understanding for the structure of the database, the database designers provide an Entity-Relationship Diagram (ERD). This diagram explains the conceptual and representational model of the data used in the database.

Authorization

Endpoint	Authorize
Auth	
POST /api/Auth/SignIn	
GET /api/Auth/Me	All Roles
Cart	
GET /api/Transaction/{uniqueCode}/Cart	
POST /api/Transaction/{uniqueCode}/Cart	
DELETE /api/Transaction/{uniqueCode}/Cart/{itemId}	
POST /api/Transaction/{uniqueCode}/Cart/Order	
Items	
GET /api/Items	
Transaction	
POST /api/Transaction	Cashier, Waiter
GET /api/Transaction/{uniqueCode}/Orders	
POST /api/Transaction/{uniqueCode}/Pay	Cashier
PUT /api/Transaction/{uniqueCode}/Orders/{orderId}/Items/{itemId}/Status	Chef, Waiter

1. Auth

POST /api/Auth/SignIn

This endpoint is designed to handle user authentication by validating the provided credentials and returning a JSON Web Token (JWT) if the credentials are valid. This token can then be used for subsequent requests to authenticate the user.

Detailed Description

- **Endpoint:** POST /api/Auth/SignIn
- **Purpose:** To authenticate a user and return a JWT for authorized access to protected resources.
- **Request Body:** The endpoint expects a JSON object containing the user's credentials:
 - **username:** The username of the user.
 - **password:** The password of the user.

Process Flow

1. **Receive Request:** The endpoint receives a POST request with the user's credentials in the request body.
2. **Validate Credentials:** The credentials are validated against the stored user data (e.g., in a database). This typically involves checking if the username exists and if the password matches the stored hash.

Hash Password Function:

```
string HashPassword(string password) {  
    var hashedBytes = SHA256.HashData(Encoding.UTF8.GetBytes(password));  
    return BitConverter.ToString(hashedBytes).Replace("-", "").ToLower();  
}
```

3. **Generate JWT:** If the credentials are valid, a JWT is generated. This token includes claims such as the user's identity and roles, and it is signed using a secret key to ensure its integrity.

4. **Return Token:** The generated JWT is returned in the response body. This token can be used by the client for subsequent requests to access protected endpoints.

This endpoint is crucial for securing the application by ensuring that only authenticated users can access protected resources. The JWT provided upon successful authentication can be used to authorize subsequent requests, maintaining a secure and efficient authentication mechanism.

Credetial Example:

Role	Username	Password
Cashier	fcerith0	Cashier123!
Waiter	odearing1	Waiter123!
Chef	darmell2	Chef123!

GET /api/Auth/Me

This endpoint is designed to retrieve the authenticated user's details based on the provided JWT. This endpoint allows users to fetch their own profile information, ensuring that they are authenticated before accessing this data.

Detailed Description

- **Endpoint:** GET /api/Auth/Me
- **Purpose:** To retrieve the authenticated user's profile information.
- **Authentication:** Requires a valid JWT in the Authorization header.

Process Flow

1. **Receive Request:** The endpoint receives a GET request with the JWT included in the Authorization header.
2. **Validate Token:** The JWT is validated to ensure it is not expired and is correctly signed.
3. **Extract User Information:** If the token is valid, the user's information is extracted from the token claims.

4. **Fetch User Details:** The user's details are fetched from the database using the information extracted from the token.
5. **Return User Details:** The user's profile information is returned in the response.

This endpoint is essential for allowing users to access their own profile information securely, ensuring that only authenticated users can retrieve their data.

2. Cart

GET /api/Transaction/{uniqueCode}/Cart

This endpoint is designed to retrieve the current items in a user's cart during an ongoing transaction. This endpoint is essential for users to view the items they have added to their cart before completing a purchase.

Detailed Description

- **Endpoint:** GET /api/Transaction/{uniqueCode}/Cart
- **Purpose:** To retrieve the current items in a user's cart for an ongoing transaction identified by a unique code.
- **Request Parameter:**
 - **uniqueCode (required):** A string representing the unique code of the transaction.

Process Flow

1. **Receive Request:** The endpoint receives a GET request with the uniqueCode as a route parameter.
2. **Query Database:** The function queries the Transactions table to find a transaction with the specified uniqueCode that is not closed.
3. **Include Related Data:** The query includes related CartItems, ordered by the time they were added and each CartItem includes its associated Item.
4. **Check Transaction:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".
5. **Return Cart Items:** If a matching transaction is found, the function returns a 200 OK response with the list of CartItems.

This endpoint is crucial for allowing users to view the items in their cart during an ongoing transaction, ensuring they can review their selections before completing the purchase.

POST /api/Transaction/{uniqueCode}/Cart

This endpoint is designed to add a new item to a user's cart during an ongoing transaction. This endpoint is essential for users to add items to their cart before completing a purchase.

Detailed Description

- **Endpoint:** POST /api/Transaction/{uniqueCode}/Cart
- **Purpose:** To add a new item to a user's cart for an ongoing transaction identified by a unique code.
- **Request Parameters:**
 - **uniqueCode (required):** A string representing the unique code of the transaction.
 - **CartItem (required):** A JSON object representing the item to be added to the cart.

Process Flow

1. **Receive Request:** The endpoint receives a POST request with the uniqueCode as a route parameter and a CartItem object in the request body.
2. **Validate Input:** The function validates the input to ensure the CartItem object is valid.
3. **Query Database:** The function queries the Transactions table to find a transaction with the specified uniqueCode that is not closed.
4. **Check Transaction:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".
5. **Add Item to Cart:** If a matching transaction is found, the function adds the new CartItem to the transaction's cart.
6. **Save Changes:** The function saves the changes to the database.
7. **Return Success:** The function returns a 201 Created response with the added CartItem.

DELETE /api/Transaction/{uniqueCode}/Cart/{itemId}

This endpoint is designed to remove an item from a user's cart during an ongoing transaction. This endpoint is essential for users to manage their cart by removing items they no longer wish to purchase.

Detailed Description

- **Endpoint:** DELETE /api/Transaction/{uniqueCode}/Cart/{itemId}
- **Purpose:** To remove an item from a user's cart for an ongoing transaction identified by a unique code.
- **Request Parameters:**
 - **uniqueCode (required):** A string representing the unique code of the transaction.
 - **itemId (required):** An integer representing the ID of the item to be removed from the cart.

Process Flow

1. **Receive Request:** The endpoint receives a DELETE request with the uniqueCode as a route parameter and the itemId as another route parameter.
2. **Validate Input:** The function validates the input to ensure the uniqueCode and itemId are valid.
3. **Query Database:** The function queries the Transactions table to find a transaction with the specified uniqueCode that is not closed.
4. **Check Transaction:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".
5. **Find Cart Item:** If a matching transaction is found, the function searches for the CartItem with the specified itemId in the transaction's cart.
6. **Check Cart Item:** If the CartItem is not found, the function returns a 404 Not Found response with the message "Item not found in cart".

7. **Remove Item from Cart:** If the CartItem is found, the function removes the item from the transaction's cart. And saves the changes to the database.
8. **Return Success:** The function returns a 204 No Content response indicating the item was successfully removed.

POST /api/Transaction/{uniqueCode}/Cart/Order

The AddItemToCartAsync endpoint in the CartController is designed to add an item to a user's cart during an ongoing transaction. This endpoint is essential for users to manage their cart by adding items they wish to purchase.

Detailed Description

- **Endpoint:** POST /api/Transaction/{uniqueCode}/Cart/Order
- **Purpose:** To add an item to a user's cart for an ongoing transaction identified by a unique code.
- **Request Parameters:**
 - **uniqueCode (required):** A string representing the unique code of the transaction.
 - **itemId (required in body):** An integer representing the ID of the item to be added to the cart.
 - **quantity (required in body):** An integer representing the quantity of the item to be added.

Process Flow

1. **Receive Request:** The endpoint receives a POST request with the uniqueCode as a route parameter and the itemId and quantity in the request body.
2. **Validate Input:** The function validates the input to ensure the uniqueCode, itemId, and quantity are valid.

3. **Query Database:** The function queries the Transactions table to find a transaction with the specified uniqueCode that is not closed.
4. **Check Transaction:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".
5. **Find Item:** The function searches for the item with the specified itemId in the Items table.
6. **Check Item:** If the item is not found, the function returns a 404 Not Found response with the message "Item not found".
7. **Add Item to Cart:** If the item is found, the function adds the item to the transaction's cart with the specified quantity. And saves the changes to the database.
8. **Return Success:** The function returns a 201 Created response indicating the item was successfully added to the cart.

3. Items

GET /api/Items

The GetItems endpoint in the ItemsController is designed to retrieve a list of categories from the database, including their related items, based on specified search criteria. This endpoint allows users to filter items within each category by their name or description and returns the filtered and ordered results.

Detailed Description

- **Endpoint:** GET /api/Items
- **Purpose:** To retrieve a list of categories and their related items, filtered by a search string.
- **Request Parameters:**
 - search (optional): A string used to filter items by their name or description. If not provided, all items are included.

Process Flow

1. **Receive Request:** The endpoint receives a GET request with an optional search query parameter.
2. **Query Database:**
 - The function queries the Categories table.
 - It includes related Items that match the search criteria (i.e., items whose name or description contains the search string).
 - The items within each category are ordered by their name.
 - The categories themselves are ordered by their name.
3. **Filter Items:** The function filters the items within each category where the item's name or description contains the search string.
4. **Retrieve Results:** The results are retrieved as a list of categories, each containing the filtered and ordered items.

5. **Return Results:** The function returns an action result containing the list of filtered and ordered categories.

This endpoint is essential for providing users with the ability to search and browse items within categories, enhancing the user experience by allowing them to find specific items quickly and efficiently.

4. Transaction

POST /api/Transaction

This endpoint is designed to initiate a new transaction for a specific table in a restaurant setting. This endpoint is essential for starting a new transaction, ensuring that each transaction has a unique identifier and that no table has more than one open transaction at a time.

Detailed Description

- **Endpoint:** POST /api/Transaction
- **Purpose:** To create a new transaction for a specific table.
- **Request Parameters:**
 - **tableNumber** (required in form data): A string representing the number of the table for which the transaction is to be created.

Process Flow

1. **Receive Request:** The endpoint receives a POST request with the tableNumber in the form data.
2. **Query Database for Table:** The function queries the Tables table in the database to retrieve the ID of the table with the specified tableNumber.
3. **Check Table Existence:** If no matching table is found, the function returns a 404 Not Found response with the message "Table not found".
4. **Check for Open Transaction:** If a matching table is found, the function checks if there is already an open transaction for that table by querying the Transactions table.
5. **Handle Existing Open Transaction:** If an open transaction exists, the function returns a 400 Bad Request response with the message "Table already has an open transaction".
6. **Generate Unique Code:** If no open transaction exists, the function generates a unique code for the new transaction. The unique code must consist of 4 characters, including both letters and numbers. It ensures the uniqueness of the code by checking the

database to confirm that no open transaction has the same code. The function continues generating codes until a unique one is found.

7. **Create New Transaction:** Once a unique code is generated, the function creates a new transaction with this code and associates it with the table. And saves the new transaction to the database.
8. **Return Success:** If the table is successfully found and the transaction is created, the function returns a 200 OK status with the generated unique code.

GET /api/Transaction/{uniqueCode}/Orders

This endpoint is designed to retrieve all orders associated with a specific transaction identified by a unique code. This endpoint is essential for fetching the details of all orders linked to a particular transaction, allowing users to view the items ordered under that transaction.

Detailed Description

- **Endpoint:** GET /api/Transaction/{uniqueCode}/Orders
- **Purpose:** To retrieve all orders associated with a specific transaction identified by a unique code.
- **Request Parameters:**
 - **uniqueCode** (required in URL path): A string representing the unique code of the transaction for which the orders are to be retrieved.

Process Flow

1. **Receive Request:** The endpoint receives a GET request with the uniqueCode in the URL path.
2. **Query Database for Transaction:** The function queries the Transactions table in the database to retrieve the transaction with the specified uniqueCode.
3. **Check Transaction Existence:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".

4. **Retrieve Orders:** If a matching transaction is found, the function queries the Orders table to retrieve all orders associated with the transaction's ID.
5. **Return Orders:** The function returns a 200 OK status with the list of orders if the transaction is found and orders are retrieved successfully.

POST /api/Transaction/{uniqueCode}/Pay

This endpoint is designed to mark a specific transaction as paid or closed. This endpoint is essential for closing a transaction and recording the payment details, ensuring that the transaction is no longer open and can be finalized.

Detailed Description

- **Endpoint:** POST /api/Transaction/{uniqueCode}/Pay
- **Purpose:** To mark a specific transaction as paid.
- **Request Parameters:**
 - **uniqueCode** (required in URL path): A string representing the unique code of the transaction to be paid.

Process Flow

1. **Receive Request:** The endpoint receives a POST request with the uniqueCode in the URL path.
2. **Query Database for Transaction:** The function queries the Transactions table in the database to retrieve the transaction with the specified uniqueCode.
3. **Check Transaction Existence:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".
4. **Check if Transaction is Already Paid:** If a matching transaction is found, the function checks if the transaction is already marked as paid. If it is, the function returns a 400 Bad Request response with the message "Transaction is already paid".

5. **Mark Transaction as Paid:** If the transaction is not already paid, the function updates the transaction's status to paid and records the payment details. And saves the updated transaction to the database.
6. **Return Success:** If the transaction is successfully updated and marked as paid, the function returns a 200 OK status with a confirmation message.

PUT /api/Transaction/{uniqueCode}/Orders/{orderId}/Items/{itemId}/Status

This endpoint is designed to update the status of a specific item within an order associated with a transaction. This endpoint is essential for managing the status of individual items in an order, allowing for updates such as marking an item as prepared, served, or canceled.

Detailed Description

- **Endpoint:** PUT
/api/Transaction/{uniqueCode}/Orders/{orderId}/Items/{itemId}/Status
- **Purpose:** To update the status of a specific item within an order associated with a transaction.
- **Request Parameters:**
 - uniqueCode (required in URL path): A string representing the unique code of the transaction.
 - orderId (required in URL path): A string representing the ID of the order.
 - itemId (required in URL path): A string representing the ID of the item within the order.
 - status (required in request body): A string representing the new status of the item.

Process Flow

1. **Receive Request:** The endpoint receives a PUT request with the uniqueCode, orderId, and itemId in the URL path, and the status in the request body.

2. **Query Database for Transaction:** The function queries the Transactions table in the database to retrieve the transaction with the specified uniqueCode.
3. **Check Transaction Existence:** If no matching transaction is found, the function returns a 404 Not Found response with the message "Transaction not found".
4. **Query Database for Order:** If a matching transaction is found, the function queries the Orders table to retrieve the order with the specified orderId and ensure it belongs to the transaction.
5. **Check Order Existence:** If no matching order is found, the function returns a 404 Not Found response with the message "Order not found".
6. **Query Database for Order Item:** If a matching order is found, the function queries the OrderItems table to retrieve the item with the specified itemId and ensure it belongs to the order.
7. **Check Order Item Existence:** If no matching item is found, the function returns a 404 Not Found response with the message "Item not found".
8. **Update Item Status:** If the item is found, the function updates the status of the item with the new status provided in the request body. And saves the updated item status to the database.
9. **Return Success:** If the item status is successfully updated, the function returns a 200 OK status with a confirmation message.